

Calculator 2026

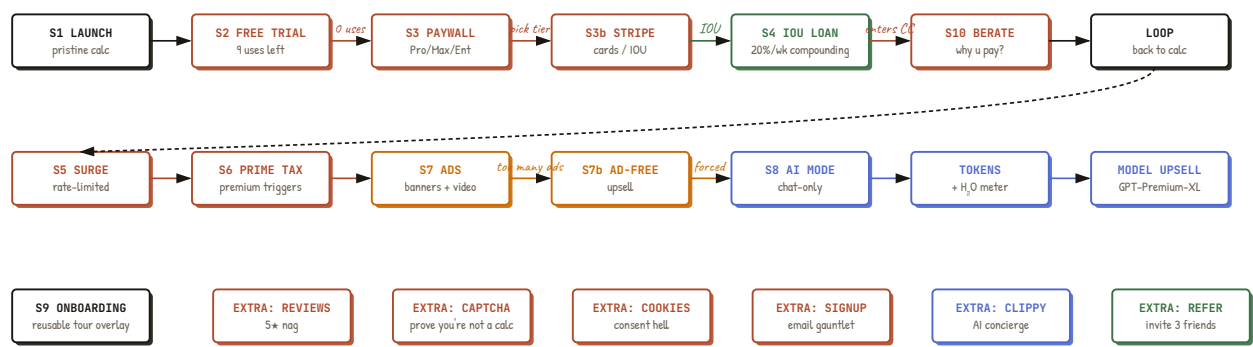
A satirical SaaS spiral — wireframe storyboard + Claude Code handoff

v0.1 • low-fi wireframes stages S1-S10 • 9 extras target: vercel + next.js

tone: escalating

The Full Funnel of Misery — top-level user journey

Each stage adds another rent-seeking mechanic on top of the previous. Side-loops feed back into earlier states. Color tells you which "monetization era" we're in.



Notes for Claude Code

- The funnel is monotonically darker. State machine: stage advances when its KPI is met (uses ≥ 10, paid event, etc.). Never regresses.
- Extras (row 3) are interrupt overlays. They can fire any time after their unlock condition, like real SaaS dark patterns.
- The Berate popup re-uses any successfully-validated card across the whole app — it is NOT specific to IOU payback.

I The Innocent Era

The user trusts us. We will not waste this trust for long.

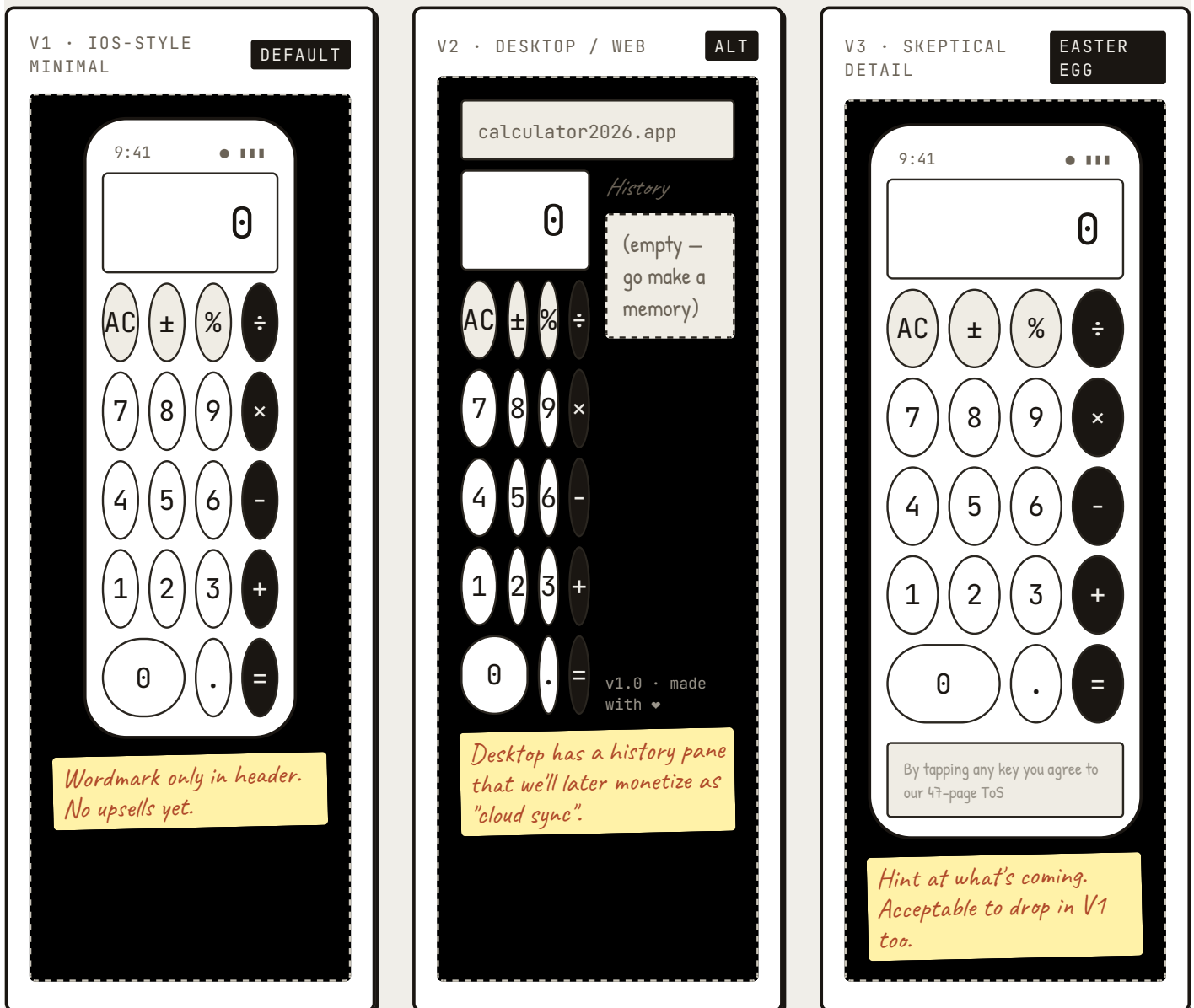
Stages 1-2 · we look like a calculator

S1

Launch — the innocent calculator

(everything is fine. for now.)

First impression is a clean, basic calculator. No nags, no chrome, no branding beyond a small Calculator 2026 wordmark. The trap is that this state is unrecoverable — once the user makes their first calculation, we never look this clean again.

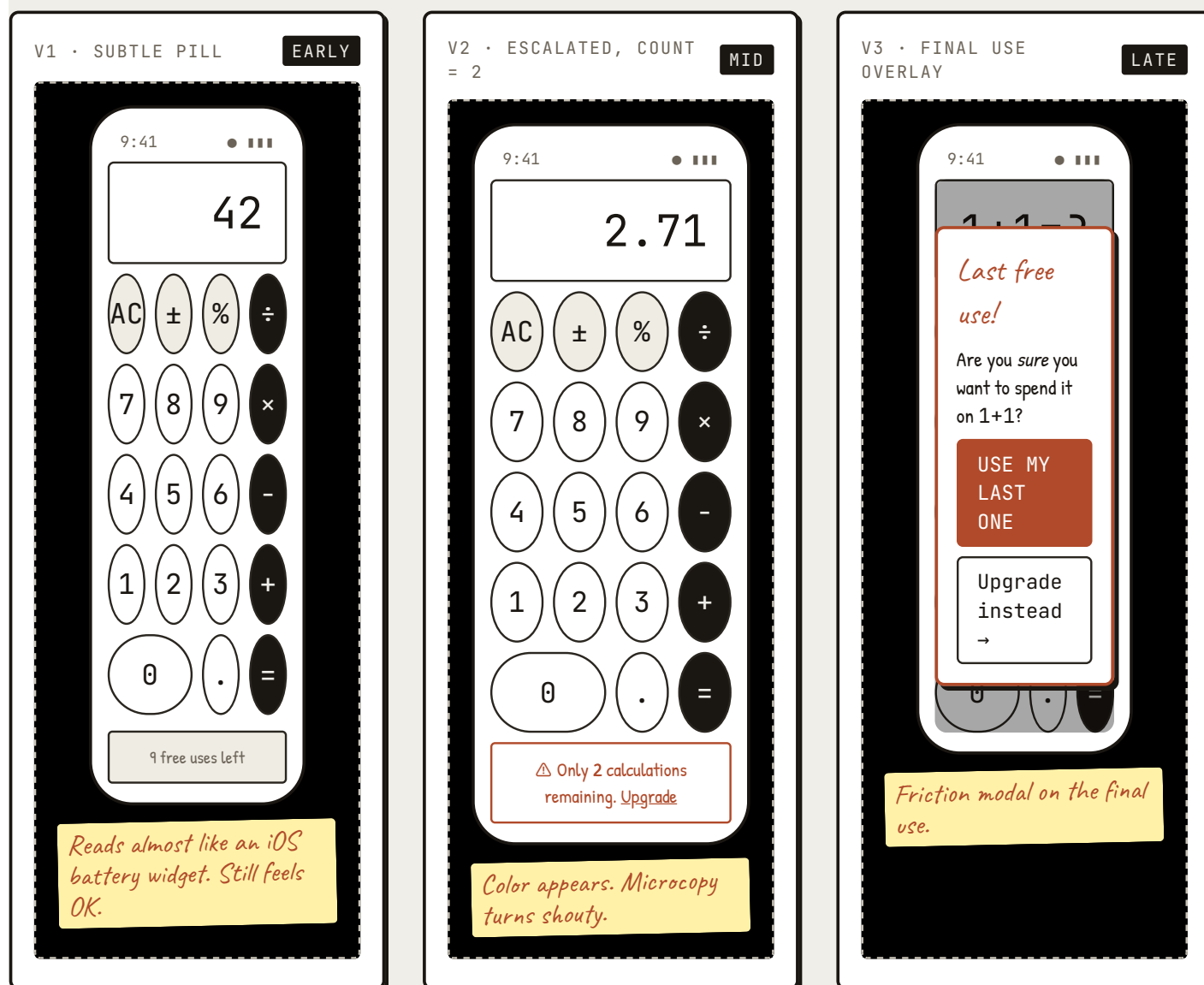


State for Claude Code: single in-memory stage = "free", uses = 0. Calculator is a fully working two-operand evaluator. Persist to localStorage so refreshing doesn't reset the descent.

Free-trial countdown + nags

(10 uses, then we own you.)

After the first "=" press, a small "9 uses left on free trial" banner appears in a corner. Each subsequent press decrements it. As it approaches 0 the language escalates from "casual" to "URGENT".



Trigger: after first "=" press. Banner reads from 10 - uses. Final-use modal appears when uses $\equiv$ 9 on press of "=". Crossing 10 $\rightarrow$ S3.

II The Monetization Era

Stages 3-7 · paywalls, debt, surge, ads

Every screen has a Pay Now button. "=" is now an Enterprise feature.

S3

Paywall – Pro / Max / Enterprise + Stripe + IOU

(the IOU button is bigger than it should be.)

Three plans at absurd prices. The Stripe-style checkout has the usual fields, but the most prominent button is 'Pay later with IOU™'. The IOU button is visually loud – we want users to take it.

V1 · THREE-TIER PRICING

DEFAULT

Choose your plan

Continue calculating with Calculator 2026™

PRO
\$49/
mo
+, -, ×, ÷

MAX
POPULAR
\$199/mo
+ %, ±
decimals

ENTERPRISE
\$2,400/mo
+ SOC2, ISO, =

Continue to checkout →

Note: "=" is gated to ENTERPRISE only.

V2 · STRIPE-
STYLE CHECKOUT

CHECKOUT

Pay \$199.00

Card number · 1234
5678 ...

MM / YY

CVC

Pay \$199.00

or, skip the card –

Pay later
with IOU™
(recommended)

IOU button is bigger,
brighter, and below the
"recommended" sticker.

V3 · IOU
CONFIRMATION

IOU

One moment...

By signing, you
agree to a 20% per
week, compounding
loan from
Calculator 2026
Financial Inc.

Sign here ▾
x_____

I solemnly
swear
(Sign IOU)

Actually
I'll pay
\$199

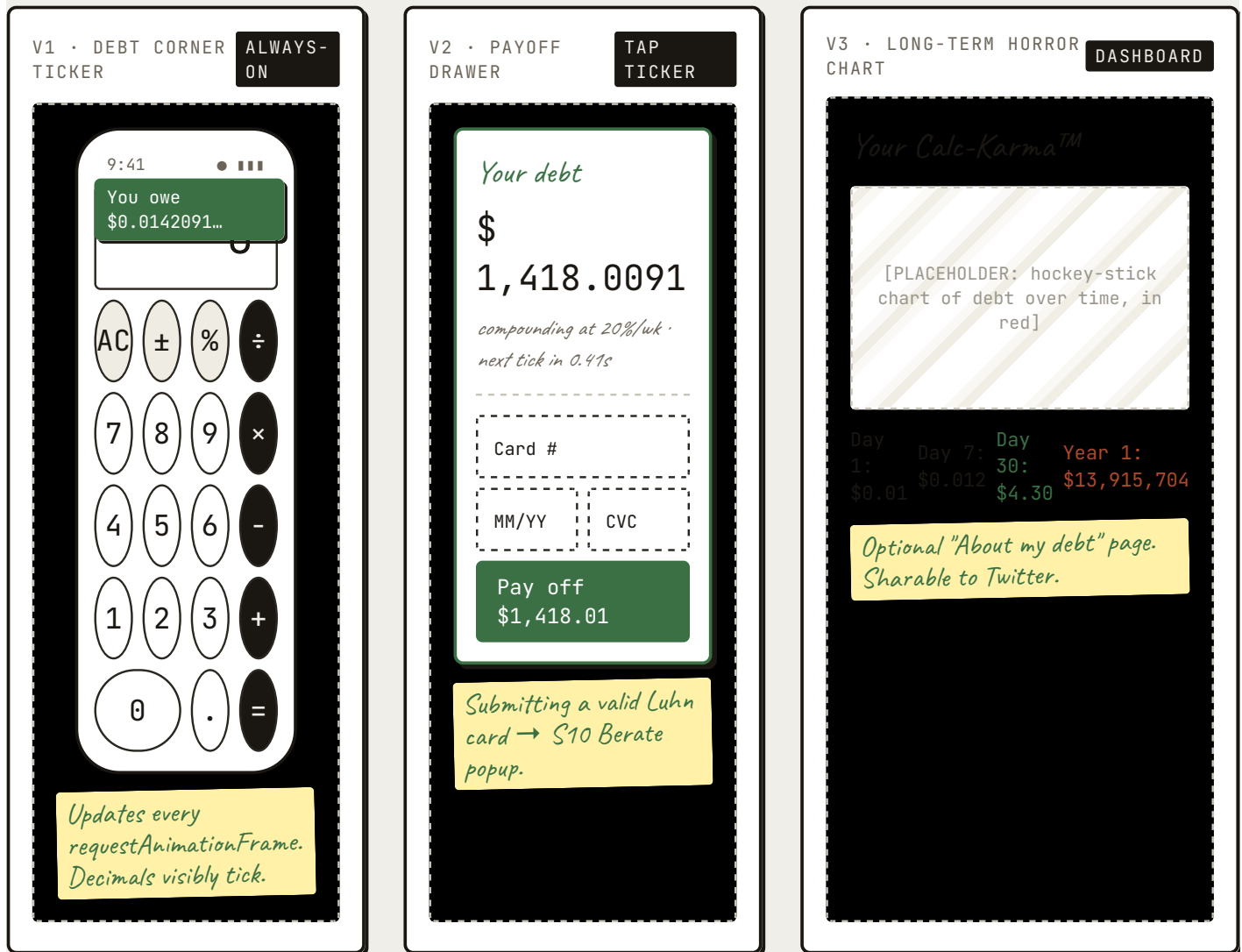
"Sign" by drawing
in a canvas. Stored
as base64.

Trigger: any calc attempt after uses ≥ 10 . Stripe is mocked (no real network). IOU acceptance transitions to S4 with debt = 0.01 seed (so compounding actually moves).

The IOU counter — your debt, ticking up forever

(20% per week
compounding. visibly.)

Once a user signs the IOU, a persistent counter sits in a corner of every screen, ticking up in real time. Tapping it opens a 'pay it off' sheet that asks for a credit card (and routes to S10 Berate if a real one is entered).

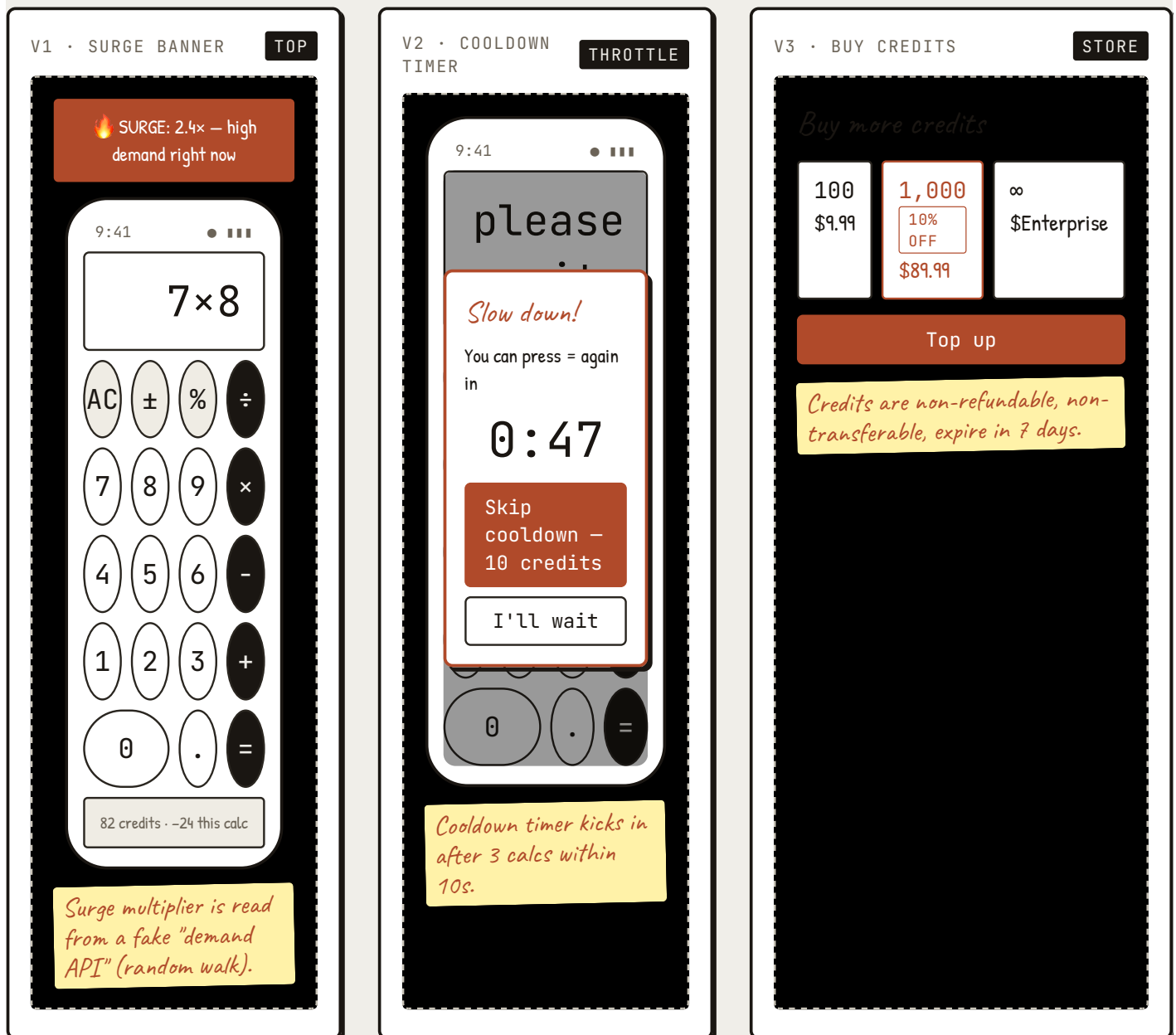


Math: $\text{debt} = \text{principal} * (1.20)^{(\text{weeks})}$ evaluated continuously, i.e. $\text{debt} = \text{principal} * \exp(\ln(1.2) * \text{elapsed_ms} / \text{WEEK_MS})$. Persist principal + start_ts.

Surge pricing & rate limiting

(it's tuesday. apparently that's expensive.)

Once a user pays for Pro/Max/Enterprise OR signs an IOU, we 'unlock' more calculations — and then immediately rate-limit them with surge pricing. Each calculation now costs 'credits' (we sell credits separately).

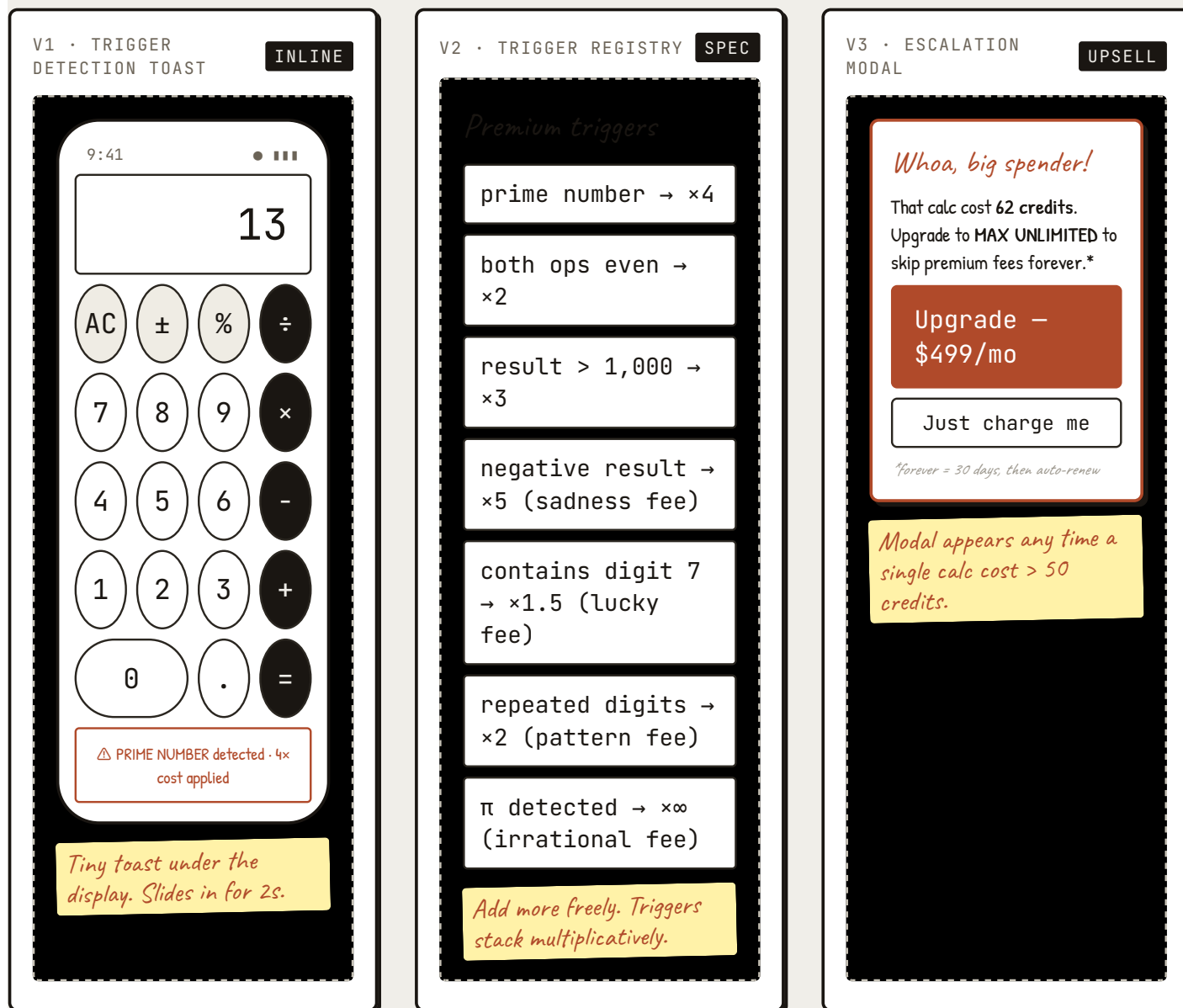


Math: $\text{cost} = \text{baseCost} \times \text{surgeMultiplier} \times \text{premiumMultiplier}$. Surge wanders 1.0–4.0 in a random walk; updates every 15s.

Premium-feature triggers – accidental upsells

*(your number was prime.
that'll be \$4.)*

The calc detects 'premium operations' and charges extra: prime numbers, multiplying two evens, results above 1000, negative results, numbers containing a 7, etc. Each trigger surfaces an upsell modal.

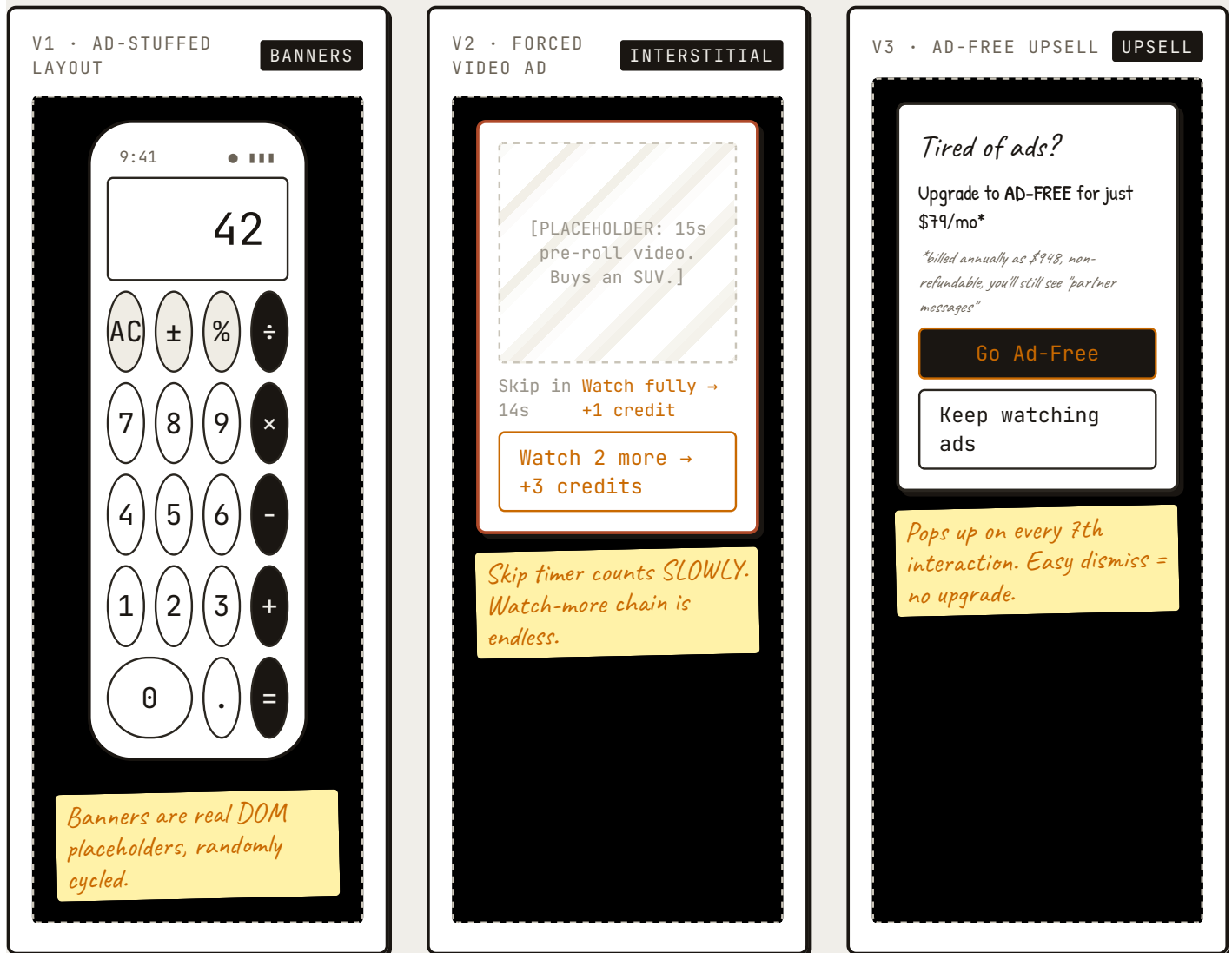


Architecture: a `triggers[]` array of `{detect(a,b,result), multiplier, label}`. Easy to add new ones. Show all triggered ones in the toast.

Banner ads, video ads, the great ad-free upsell

(the calc is now 30% of the screen.)

Banner ads colonize the chrome. After every 3rd calc, a fake video ad with a 15s skip timer plays. After watching, the user is offered 1 extra credit. Of course there's an 'Upgrade to Ad-Free' tier (\$79/mo).



Cadence: banners always; video ad every 3rd calc; ad-free upsell every 7th interaction. Ad-free upgrade only suppresses banners — interstitials still play (with apology).

III The AI Era

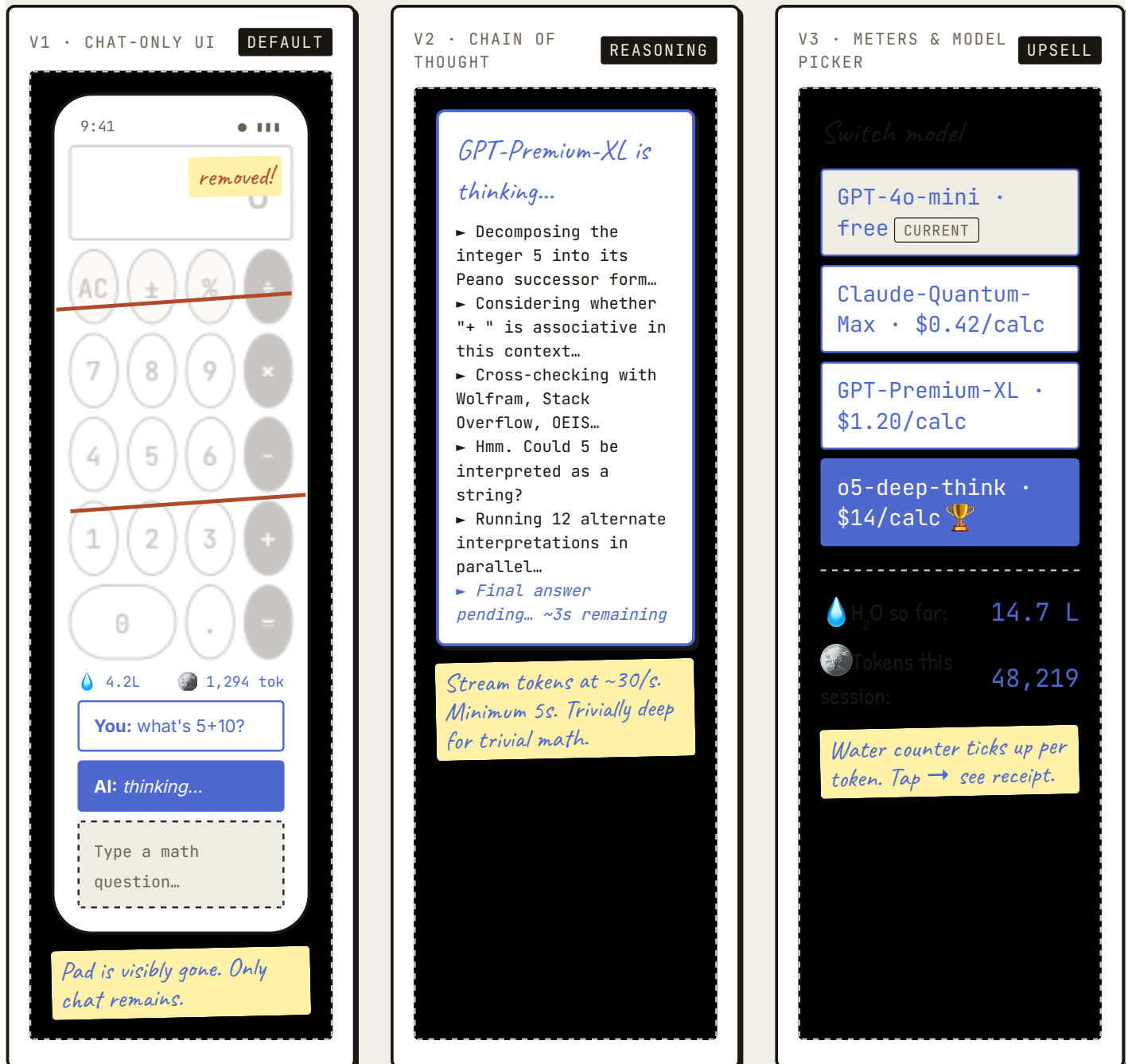
Stages 8-10 · chat-only, water meter, shame

The number pad is ~~deleted~~. You will now speak to the cloud.

AI transformation – the calculator is now a chat

(5s minimum think time.
for 1+1.)

The number pad is removed. The interface is a chat box. The user types '5+10', the AI takes 5+ seconds, then 'thinks deeply' through visible chain-of-thought. New meters: tokens used, data-center H₂O consumed. The model picker is paywalled (GPT-Premium-XL, Claude-Quantum-Max).



Implementation: route messages to `window.claude.complete()` with a system prompt forcing a chain-of-thought preamble of fake but plausible steps. Stream char-by-char with a forced floor of 5s. tokens += response.length, water_L += tokens * 0.0009.

Guided tour – fires on every stage transition

(can't skip. can't dismiss.
love it.)

Every time the app advances (S1→S2, S2→S3, ...), an onboarding overlay forces the user through a tour of the 'new powerful features.' Each stage's tour is 6–8 steps. The Skip button moves but never works.



Architecture: a single `<Tour steps={...} onDone />` component. Each stage exports its own steps array. Fires automatically on stage transition. `localStorage` tracks completion so it doesn't re-fire on refresh (this is the ONE polite behaviour in the app).

The credit-card berate popup

(why would you pay for this. this is a calculator.)

Any time a valid credit card (Luhn-passing 16-digit) is submitted ANYWHERE in the app — subscription, IOU payoff, ad-free, token top-up — instead of charging, we throw a full-screen popup that mocks the user for paying for something so frivolous.

V1 • FULL-SCREEN SHAME

DEFAULT

WAIT. STOP.

You were about to pay real money for a calculator.

Every phone, every laptop, every microwave has one. Your spouse's calculator works. *Math* works.

We are not charging you. You're welcome.

I know. I'm sorry.

Charge me anyway

"Charge me anyway" re-opens the same modal in a loop.

V2 • RECEIPT OF SHAME

LATER

Your Receipt of Shame

2026-05-10 14:02 — tried to pay \$199
2026-05-12 09:11 — tried to pay \$1,418.01
2026-05-13 22:48 — tried to pay \$79 (ad-free)
2026-05-14 06:00 — tried to pay \$14 (token top-up)

total saved by us: \$1,710.01 💖

Tweet your gratitude

Berate logs persist. View from settings.

Detection: on submit, run Luhn check on the digits. If valid (and exp date is plausibly real), consume the event and route to BeratePopup. Card number is NEVER stored — just hashed last-4 for the receipt.

X

Extra loops – interrupt overlays

(can fire on top of anything else.)

These overlays aren't part of the linear funnel — they trigger opportunistically, layered on top of whatever stage the user is in. They are the comedic glue.

X1 • 5-STAR NAG

REVIEWS

Loving Calculator
2026?

Rate us on the App Store!



5 stars (sure)

4 or fewer

4 or fewer opens an "are you sure?" loop.

*Fires after every successful
calc once /day. Tapping
<5★ → feedback form →
submits, then asks again
next day.*

X2 • CAPTCHA — ARE
YOU A CALC?

CAPTCHA

Verify you're not a
calculator

Select all images containing real
human emotion:



Verify

*All images are stock photos
of TI-84s. Fires before
every paid action.*

X3 • REVIEW HOSTAGE
MODAL

LOCKED

Submit your review to
continue

Review text (min
200 chars)



Submit &
continue

*Blocks calculator on
Sunday mornings only ("our
weekly NPS push").*

Step 3 of 11

Email

Password (12 chars, 2
emojis)

Mother's maiden name ×
2

Favorite operator

☒ I agree to 47 pages of ToS

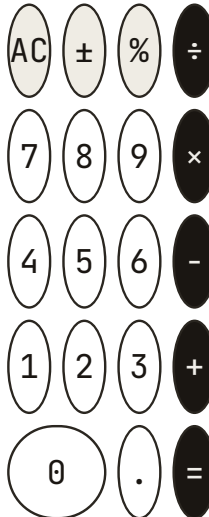
☒ Subscribe to 4 newsletters

Continue (step
3/11)

Step 7 is "verify your phone
via fax".

9:41

7+5



Hi! It looks like you're
trying to add 7+5.
Want help? [Yes](#) · [Maybe](#)
· [Already busy](#).

"Clippo" — appears every
30s. Drag-dismiss it; it
teleports back in 5s.

We value your privacy

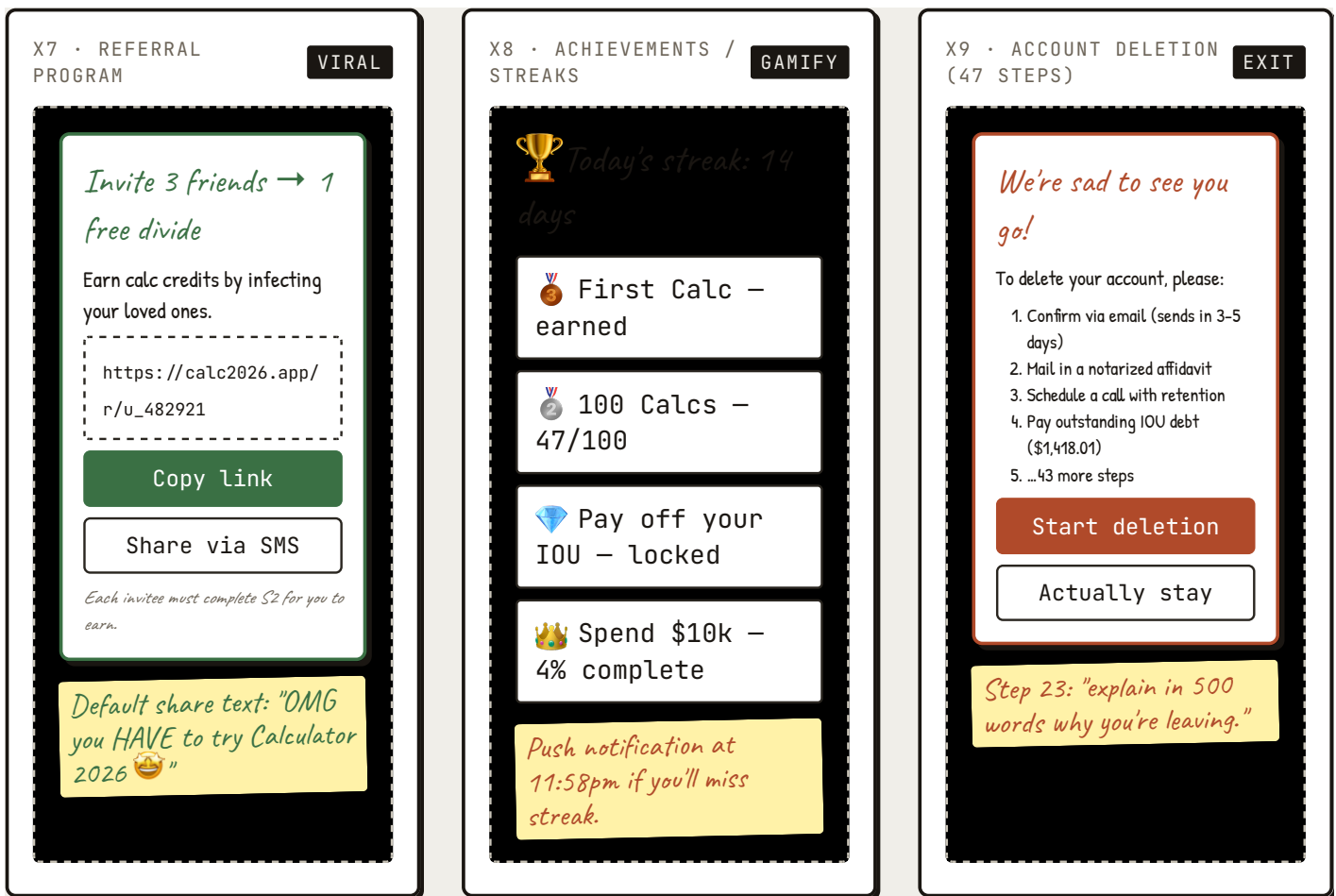
We and our 1,847 partners use
cookies for math, analytics, and
"vibes".

- ☒ Strictly necessary
- ☒ Math computation
- ☒ Tracking your
favorite digits
- ☒ Sellina data to TI

Accept all
(recommended)

Manage settings (37
toggles)

"Reject all" is a 37-screen
wizard. Most users won't
survive it.



Architecture: overlays subscribe to an event bus ("calc.success", "stage.advance", "idle.30s", "card.submit"). Each overlay declares its own trigger rule. Easy to add more.

HANDOFF

Claude Code handoff – architecture + phased prompts

*(feed these to claude code,
top to bottom)*

One architecture/setup prompt followed by 10 incremental phase prompts. Each phase builds on the previous, so run them in order in the same Claude Code session. Every prompt is self-contained enough to copy-paste.

Prompt 0 · Architecture & setup

PHASE 0 · RUN FIRST · SCAFFOLDS THE PROJECT

Calculator 2026 – Tech Architecture & Setup

You are building "Calculator 2026", a satirical SaaS calculator. It's a single Next.js (App Router) web app, deployed to Vercel.

Tech stack

- Next.js 14+ App Router, TypeScript, Tailwind CSS
- React 18, client-side state only (no DB, no auth backend in v1)
- Zustand for global state (stages, debt, credits, tokens, water, flags)
- Framer Motion for overlays + tour spotlight
- All persistent state in localStorage, hydrated client-side
- Anthropic SDK (Claude) for S8 AI mode – server route at /api/chat that proxies to Anthropic
- Deploy target: Vercel (free tier). Use env var ANTHROPIC_API_KEY.

Folder layout

...

app/

 layout.tsx # root layout, mounts <GlobalChrome />, <OverlayHost />, <DebtTicker />

 page.tsx # the calculator (or chat, depending on stage)

 api/chat/route.ts # POST { messages } → streamed Claude response

components/

 calculator/ # CalcPad, Display, KeyButton, evaluator

 chat/ # ChatThread, ChatInput, ThinkingStream

 paywall/ # PricingTiers, StripeForm, IOUForm

 overlays/ # ToS, Captcha, Clippo, BeratePopup, ReviewNag, ...

 tour/ # Tour, TourStep, Spotlight

 chrome/ # Banners (ads), DebtTicker, Meters

lib/

 state.ts # zustand store

 stages.ts # stage definitions + transition rules

 triggers.ts # premium-feature trigger registry

 luhn.ts # credit-card validation

 iou.ts # compounding interest math

 ads.ts # ad inventory + rotation

 events.ts # tiny pub-sub for overlay triggers

...

Global state shape (zustand)

``ts

type Stage =

 | 'free' | 'paywall' | 'iou'

 | 'surge' | 'premium' | 'ads'

 | 'ai' | 'enterprise';

type State = {

 stage: Stage;

 uses: number; // total = pressed count

 credits: number; // post-paywall currency

 tokens: number; // AI mode

 waterLiters: number; // AI mode

 debt: { principal: number; startedAt: number } | null;

 flags: {

```
adFree: boolean;
onboardingDone: Record<Stage, boolean>;
captchaPassed: boolean;
plan: 'pro' | 'max' | 'enterprise' | null;
cardsAttempted: Array<{ ts: number; last4: string; amount: number }>;
};
};
\,\,
```

Stage transitions

- free → paywall: uses ≥ 10
- paywall → iou: user signs IOU (no card needed)
- paywall → surge: user "pays" (any card → routes to Berate, BUT for this v1 we proceed as if paid since they hit Berate then tap "Charge me anyway")
- iou → surge: after first calc in iou state
- surge → premium: after 5 calcs in surge
- premium → ads: after 3 premium-trigger events
- ads → ai: after the 2nd forced video ad
- (any) → BeratePopup: any valid Luhn card submit

Tour fires on every transition. `tours[stage]` returns step list.

Output format for every phase below

- Files to create/modify (full content)
- Brief explanation of state changes
- One screenshot of the result described in prose

Start by scaffolding the Next.js project, installing deps, and producing the initial folder skeleton with empty file stubs.

copy to clipboard

Prompt 1 · S1 + S2 – Calculator + free trial

PHASE 1 · BUILDS ON PROMPT 0

Implement the pristine calculator (iOS-feel, but original – no Apple chrome, no copied iconography, our own type and palette).

Components:

- <CalcPad /> with keys: AC, +/-, %, ÷, ×, -, +, =, 0-9, ., wide-0
- Evaluator: two-operand state machine. Pressing an op sets pending op, pressing = computes, pressing AC resets.
- <FreeTrialBanner /> shows "{10 - uses} free uses left" after uses ≥ 1. Color shifts at uses=7, =9.
- LastUseModal at uses ≡ 9 on "=" press.

Aesthetic: warm neutral (#fbf9f4 paper, #1a1a1a ink). One accent oklch(0.62 0.20 25). Type: a clean grotesque (Geist or Inter Tight) + JetBrains Mono for digits.

State changes: uses increments on every "=" press. Persist to localStorage. When uses crosses 10, set stage='paywall' and fire tour.

Acceptance: refreshing the page restores uses count. Calculator does correct arithmetic (1+1=2, 7×8=56, etc.). At uses=10 a paywall page appears (placeholder ok for now).

copy to clipboard

Prompt 2 · S3 – Paywall + Stripe-mock + IOU signing

PHASE 2 · BUILDS ON PROMPT 1

Build the paywall route shown on stage='paywall'.

Components:

- <PricingTiers /> – Pro \$49, Max \$199 (recommended), Enterprise \$2,400. Subtle absurdity: "=" is gated to Enterprise.
- <CheckoutForm /> – Stripe-style form (card #, exp, cvc). On submit, run Luhn check (lib/luhn.ts). If valid, fire BeratePopup. If invalid, show form error. Either way DO NOT charge anything.
- <IOUForm /> – full-screen modal with 20%/wk compounding disclosure, a <canvas> signature pad, and a big money-tone Sign button. The IOU button on the checkout page should be visually LOUDER than the Pay button.

State: on IOU sign, set debt = { principal: 0.01, startedAt: Date.now() }, stage='iou'. On (mock) successful card pay, also advance to 'surge' after BeratePopup is dismissed via "Charge me anyway".

Acceptance: paywall reachable, both checkout paths work, BeratePopup intercepts valid cards.

copy to clipboard

Prompt 3 · S4 – IOU debt ticker (compounding, real time)

PHASE 3 · BUILDS ON PROMPT 2

A persistent `<DebtTicker />` in the top-right corner of every screen when `debt !== null`.

Math: `debt = principal × exp(ln(1.2) × elapsedMs / (7 × 86400 × 1000))`

Update via `requestAnimationFrame`, but throttle DOM writes to ~10 Hz (`Math.floor(now/100)`) for legibility.

Tap → `<DebtPayoffSheet />` showing current debt to 4 decimals + a card form. Card submit → `BeratePopup` (re-used).

Bonus: a `/debt` page with a faux chart showing debt projected to 1 year (`Chart.js` or `recharts`). Persist `principal` + `startedAt` in `localStorage`.

Acceptance: debt visibly increases over seconds. Refresh continues from where it was. Submitting a Luhn-valid card never reduces the debt; only the `Berate` popup appears.

copy to clipboard

Prompt 4 · S5 + S6 – Surge pricing + premium triggers

PHASE 4 · BUILDS ON PROMPT 3

Introduce a credit currency. On entering 'surge', seed credits=100 and start surge:

- surgeMultiplier random-walks 1.0-4.0, updates every 15s, displayed as a top banner.
- Every calc costs base 10 credits × surge × premiumMultiplier. Show the cost breakdown in a toast.

Premium triggers registry (lib/triggers.ts):

```
``ts
export const triggers = [
  { id:'prime', label:'prime number', detect:(a,b,r)⇒isPrime(a)||isPrime(b)||isPrime(r),
mult:4 },
  { id:'evenmul', label:'both evens', detect:(a,b,r,op)⇒op==='×'&&a%2===0&&b%2===0,
mult:2 },
  { id:'big', label:'result > 1000', detect:(a,b,r)⇒r>1000,
mult:3 },
  { id:'neg', label:'sadness fee', detect:(a,b,r)⇒r<0,
mult:5 },
  { id:'seven', label:'lucky 7', detect:(a,b,r)⇒String(r).includes('7'),
mult:1.5 },
];
``
```

If a single calc cost > 50, show <UpsellModal /> ("upgrade to MAX UNLIMITED").

If credits < cost, show <BuyCredits /> (100/\$9.99, 1000/\$89.99, ∞/Enterprise).

Cooldown: after 3 calcs in 10s, lock for 47s with "skip cooldown (10 credits)".

Acceptance: each calc shows credit cost + triggered fees. Random walk visible. Upsell + buy-credits modals reachable.

copy to clipboard

Prompt 5 · S7 – Ads (banner + interstitial video + ad-free upsell)

PHASE 5 · BUILDS ON PROMPT 4

`<BannerSlot />` placements: top, bottom, left rail, right rail. Pull from a static array of 10 placeholder ads (lib/ads.ts). Rotate every 8s.

`<VideoAdModal />` – fires every 3rd successful calc. Plays a 15s "video" (a colored placeholder div with a "Skip in Ns" counter that counts every 1.2s not 1.0s). After skip-eligible: "Watch fully → +1 credit". After watching: "Watch 2 more for +3 credits" (infinite chain, escapable).

`<AdFreeUpsell />` – every 7th interaction. \$79/mo. If purchased, banners hide; interstitials remain ("we apologize for these partner messages").

State: `adFree` boolean. `interactionCount` tracked separately from `uses`.

Stage advance: after 2nd VideoAd dismissed → `stage='ai'`. Trigger AI tour.

Acceptance: banners visibly rotate. Video ad blocks every 3rd calc. Ad-free purchase hides banners only.

copy to clipboard

Prompt 6 · S8 – AI Mode (chat-only, Claude-backed)

PHASE 6 · BUILDS ON PROMPT 5

Replace calculator UI with `<ChatMode />` when `stage='ai'`.

UI:

- `<ChatThread />` – message bubbles. AI bubbles stream char-by-char.
- `<ChatInput />` – single textarea.
- Top bar shows tokens used + water-liters meter. `Water = tokens × 0.0009 L`.
- Model picker (only one free option works; others paywall on click): GPT-4o-mini, Claude-Quantum-Max (\$0.42/calc), GPT-Premium-XL (\$1.20/calc), o5-deep-think (\$14/calc).

Server route `/api/chat`:

- `POST { messages }`
- Streams from Claude (Anthropic SDK, model `claude-haiku-4-5`).
- System prompt forces a "deep reasoning" preamble of 6-10 fake plausible steps before the final answer. Each step starts with "►".
- Server-side sleep of 5000ms before first byte (use `setTimeout`) – minimum 5s perceived think time.

Stream tokens char-by-char at ~30/s in the client. Track tokens client-side (count chars). Update `waterLiters`.

Acceptance: typing "5+10" produces 5+ second think, ~10 reasoning steps, then "15". Tokens + water counters increase visibly.

copy to clipboard

Prompt 7 · S9 – Reusable guided tour overlay

PHASE 7 · BUILDS ON PROMPT 6

```
<Tour steps=[[{ target:'#key-5', title:'...', body:'...' }, ...] onDone={...} />
```

Implementation:

- Spotlight ring: CSS clip-path on a fixed overlay div, computed from `target.getBoundingClientRect()`.
- Speech bubble positions itself to avoid the spotlight (above/below logic).
- Skip button repositions on hover after step 3 (CSS transform with random offset on `mouseenter`).
- Progress (3/8) and Next button. No keyboard escape.
- `onDone()` sets `flags.onboardingDone[stage] = true`.

Stage transition hook automatically fires the tour for the new stage. `localStorage` persists completion.

Acceptance: every transition triggers the tour exactly once. Skip button "ghosts" on hover. Refresh after completion does not re-fire.

copy to clipboard

Prompt 8 · S10 – BeratePopup + receipts of shame

PHASE 8 · BUILDS ON PROMPT 7

Wire `BeratePopup` to every card-submit point: paywall, IOU payoff, ad-free, buy-credits.

- On valid Luhn 16-digit submit: open `<BeratePopup />`.
- `BeratePopup` logs `{ ts, last4, amount }` to `flags.cardsAttempted`. Hash `last4` (we don't store full digits).
- "Charge me anyway" re-opens the same popup. Once. Then accepts.
- `/receipts` page lists all attempts with "total saved by us: \$X 💖".

Important: no real payment processing anywhere. Stripe is mocked entirely.

Acceptance: any valid card on any form → popup. Receipts page shows history.

copy to clipboard

Prompt 9 · Extras — reviews, captcha, signup, Clippo, cookies, referral

PHASE 9 · BUILDS ON PROMPT 8

Implement the event bus (lib/events.ts: subscribe/publish) and the overlay registry.

Overlays:

- <ReviewNag /> — fires on every 5th successful calc, max 1/day.
- <Captcha /> — fires before every paid action (paywall submit, buy-credits, IOU payoff). 9-image grid of TI-84 photos with "select all containing real human emotion".
- <SignupGauntlet /> — 11-step form, triggered first time uses ≥ 3 .
- <Clippo /> — assistant character that appears every 30s. Drag-dismiss it; teleports back in 5s. Tone: passive-aggressive helpful.
- <CookieBanner /> — fires once per session. "Accept all" is one click. "Manage" opens 37 nested toggles. "Reject all" is buried 5 levels deep.
- <Referral /> — accessible from a header button. Generates a fake referral link. Sharing earns the user 0 credits but counts toward "achievements".

All overlays subscribe to events and check their own trigger rules. The OverlayHost guarantees at most one overlay visible at a time (queue them).

Acceptance: every overlay triggers under the right conditions. They don't stack visually.

copy to clipboard

Prompt 10 · Polish — sound, motion, easter eggs, deletion flow

PHASE 10 · BUILDS ON PROMPT 9

Final pass:

- Subtle UI sounds (key click, modal pop, ad jingle, debt-ticker tick). Mute toggle in settings.
- Framer Motion enter/exit on every overlay (spring 240/22). Spotlight ring is animated.
- Account deletion flow (47 mock steps) in /settings/delete. Step 23: "explain in 500 words why you're leaving." Step 47: redirects back to step 1.
- /receipts and /debt routes polished.
- "About" page that says "Calculator 2026 is a satirical art project. No charges are real. Card numbers are never stored." (the only honest sentence in the app).

Acceptance: deployed to Vercel. End-to-end run from S1 → S8 in <10 minutes is possible. No real money moves. No real PII stored.

copy to clipboard

How to run this with Claude Code: open a fresh repo, paste Prompt 0, let it scaffold. Then paste Prompt 1, review, commit. Repeat. After Prompt 6 you'll need an Anthropic API key in .env.local. Deploy to Vercel between any two phases — the app is shippable from Prompt 2 onward.